

BOOK 01 — JAVA FUNDAMENTALS

Beginner to Professional Mastery (Telugu + English)

Series: Java Interview + Development Mastery Series **Book Number:** 01 of 24 **Java Versions**

Covered: Java 7 (baseline syntax), Java 8 (var-adjacent notes), Java 11 LTS (`var`), Java 21 LTS (text blocks preview mention) **Prerequisites:** None — start here. **Next Book:**

02_Java_00PS_Mastery_Telugu_English.md

How to Use This Book / ఈ పుస్తకాన్ని ఎలా ఉపయోగించాలి

Telugu: ప్రతి concept మొదట తెలుగులో simple గా explain చేయబడుతుంది, తర్వాత professional English లో ఇవ్వబడుతుంది. Code, examples, output ఎప్పుడూ English/Java standard లోనే ఉంటాయి (అది real-world లో మీరు రాసేది కాబట్టి). ప్రతి chapter చివర Interview Questions, Cross Questions, Exercises మరియు Revision boxes ఉంటాయి — వాటిని miss చేయకండి.

English: Every concept is explained first in simple Telugu, then in professional interview-ready English. Code and terminology stay in standard English/Java form. Each chapter ends with interview Q&A, cross-questions, tricky questions, and a 6-level exercise ladder. Do not skip the revision boxes — they are designed for fast re-reading before interviews.

Learning Objectives

By the end of this book you will be able to:

1. Explain what Java is, why it exists, and how it runs (Telugu + English).
2. Write, compile, and run Java programs confidently.
3. Use variables, data types, operators, and control flow correctly.
4. Work with arrays and methods idiomatically.
5. Understand classes, objects, constructors, `this`, `static`, `final`.
6. Understand access modifiers and packages.
7. Master `String`, `StringBuilder`, `StringBuffer`, wrapper classes, autoboxing/unboxing.
8. Use enums and nested/inner/anonymous classes.
9. Understand `equals()`, `hashCode()`, `toString()` at a foundational level.
10. Answer fundamentals-level interview questions with confidence, including cross-questions.

Table of Contents

Ch.	Title
1	Introduction to Java & Environment Setup

Ch.	Title
2	Variables & Data Types
3	Operators
4	Control Flow Statements
5	Arrays
6	Methods
7	Classes & Objects (Foundations)
8	Constructors & <code>this</code>
9	<code>static</code> and <code>final</code>
10	Access Modifiers & Packages
11	String, StringBuilder, StringBuffer
12	Wrapper Classes, Autoboxing & Unboxing
13	Enums
14	Nested, Inner & Anonymous Classes
15	Object Class Basics — <code>equals()</code> , <code>hashCode()</code> , <code>toString()</code> , Immutability Intro
16	Mini Project — Student Management Console App
—	Final Revision Notes, Cheat Sheet, Interview Bank, Revision Plans, Mastery Checklist

CHAPTER 1 — Introduction to Java & Environment Setup

Telugu Explanation

Java అనేది ఒక **high-level, object-oriented, platform-independent programming language**.

"Platform-independent" అంటే — మీరు ఒకసారి code రాస్తే (**Write Once**), అది ఏ OS లో అయినా (Windows, Linux, Mac) run అవ్వచ్చు (**Run Anywhere**) — దీన్నే **WORA principle** అంటారు. ఇది సాధ్యం అవడానికి కారణం **JVM (Java Virtual Machine)** — ఇది ప్రతి OS కి వేరుగా ఉంటుంది, కానీ compiled code (bytecode) అందరికీ common గా ఉంటుంది.

Professional English Explanation

Java is a high-level, statically-typed, object-oriented programming language designed for portability and reliability. Source code (`.java`) is compiled by `javac` into platform-independent **bytecode** (`.class`), which is then executed by the **JVM** on any platform that has a compatible JVM installed. This is the foundation of Java's "Write Once, Run Anywhere" (WORA) promise.

Simple Example — Compilation & Execution Flow

```
HelloWorld.java --(javac)--> HelloWorld.class (bytecode) --(JVM)--> Output
```

Java Code

```
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, Java!");
    }
}
```

Output

```
Hello, Java!
```

Internal Working

1. `javac HelloWorld.java` → compiler checks syntax, generates `HelloWorld.class` (bytecode).
2. `java HelloWorld` → JVM's **Class Loader** loads `HelloWorld.class` into memory.
3. **Bytecode Verifier** checks the bytecode is safe.
4. **Execution Engine** (Interpreter + JIT Compiler) executes it, starting from `public static void main(String[] args)` — the JVM entry point contract.

JDK vs JRE vs JVM (Diagram)

```
+-----+
| JDK (Java Development Kit) |
| +-----+ |
| | JRE (Java Runtime Environment) | | | |
| | +-----+ | |
| | | JVM (Java Virtual Machine) | | |
| | | - Class Loader | | |
| | | - Runtime Data Areas (Heap, Stack, etc.) | | |
| | | - Execution Engine (Interpreter + JIT) | | |
| | +-----+ | |
| | + Core Libraries (java.lang, java.util, ...) | |
| +-----+ |
| + javac, javadoc, jdb, jar (development tools) |
+-----+
```

- **JVM** — runs bytecode. Platform-specific implementation, common bytecode contract.
- **JRE** — JVM + standard libraries needed to *run* Java programs.
- **JDK** — JRE + development tools (`javac` , debugger, etc.) needed to *build* Java programs.

Deep internals (class loading phases, memory areas, GC) are covered fully in **Book 03 — JVM & Memory Management**. Here you only need the mental model above.

Real-World Example

Telugu: ఒక e-commerce backend team Linux servers me¹ద Java application run చేస్తారు, కానీ developers Windows/Mac laptops me¹ద code రాస్తారు — ఇది సాధ్యం అవడం JVM వల్లనే. English: A backend team can develop on Windows/Mac and deploy the same compiled artifact to Linux production servers without recompiling — this is WORA in daily practice.

Interview Answer

"Java is a platform-independent, object-oriented language. Source code compiles to bytecode, and the JVM executes that bytecode on any platform, which is why Java achieves 'write once, run anywhere'."

Deep Interview Answer

"The JVM abstracts away the OS and hardware. `javac` produces bytecode that conforms to the JVM spec, not to any particular OS. Any platform with a compliant JVM implementation (HotSpot, OpenJ9, etc.) can execute that same `.class` file. This separation of compilation and execution — plus JIT compilation at runtime for hot code paths — is what gives Java both portability and near-native performance."

Cross Questions

- Q: Is Java 100% platform-independent? → A: The *bytecode* is; native OS-level resources (file paths, native libraries via JNI) can still differ.

- Q: Can you run a `.java` file without compiling? → A: Since JDK 11, `java HelloWorld.java` can compile-and-run single-file source programs in one step (source-launcher), but production builds still compile first.
- Q: What is bytecode? → A: Intermediate, platform-neutral instruction set stored in `.class` files, understood by any JVM.

Tricky Questions

- Q: If JVM is platform-specific, how is Java platform-independent? → A: The *bytecode format* and language are platform-independent; each OS gets its own JVM *implementation* that knows how to translate that bytecode to native instructions for that OS.

Coding Exercise

1. Install JDK 21. Run `java -version` and `javac -version`. Write and run `HelloWorld.java`.
2. Modify the program to print your name and today's date.

Mastery Check

Explain, without notes, the journey of a `.java` file from source code to console output, naming every component involved (compiler, bytecode, class loader, execution engine).

CHAPTER 2 — Variables & Data Types

Telugu Explanation

Variable అనేది ఒక **named memory location** — value store చేయడానికి. Java **statically typed** language కాబట్టి, ప్రతి variable కి ఒక fixed **data type** ఉండాలి, అది compile time లోనే తెలియాలి. Data types రెండు రకాలు: **Primitive** (byte, short, int, long, float, double, char, boolean) మరియు **Reference/Non-primitive** (String, Arrays, Objects, Classes).

Professional English Explanation

A variable is a named storage location whose type is fixed at compile time (static typing). Java has 8 primitive types stored directly with their value, and reference types that store a reference (pointer) to an object on the heap.

Primitive Types Table

Type	Size	Default	Range (approx)
byte	1 byte	0	-128 to 127
short	2 bytes	0	-32,768 to 32,767
int	4 bytes	0	~-2.1B to 2.1B
long	8 bytes	0L	~-9.2×10 ¹⁸ to 9.2×10 ¹⁸
float	4 bytes	0.0f	~7 decimal digits precision
double	8 bytes	0.0d	~15 decimal digits precision
char	2 bytes	"	single UTF-16 character
boolean	JVM-dependent	false	true / false

Java Code

```
public class VariablesDemo {
    public static void main(String[] args) {
        int age = 28;
        double salary = 75000.50;
        char grade = 'A';
        boolean isEmployed = true;
        String name = "Ravi";        // reference type
        var city = "Hyderabad";      // Java 11+ local type inference

        System.out.println(name + " is " + age + " years old, earns " + salary
            + ", grade " + grade + ", employed=" + isEmployed + ", lives in " + city);
    }
}
```

Output

```
Ravi is 28 years old, earns 75000.5, grade A, employed=true, lives in Hyderabad
```

Internal Working / Memory View

- Primitives declared as local variables live on the **Stack**, directly holding their value.
- `String name = "Ravi"` — the reference variable `name` lives on the Stack; the actual `String` object lives in the **String Pool** (a special region inside the Heap).
- `var` is **not** a dynamic type — it is compile-time type inference; the compiler still fixes the type (`String` here) permanently.

```
STACK                                HEAP
+-----+                          +-----+
| age = 28 |                        | String Pool          | |
| salary  |                        | "Ravi"  -----+      |
| grade='A'|                        | "Hyderabad" |      |
| isEmployed|                      +-----+-----+
| name  ----+----->--+
| city  ----+----->("Hyderabad")
+-----+
```

Real-World Example

Telugu: Banking app లో `accountBalance` ని `double` బదులు `BigDecimal` వాడతారు — ఎందుకంటే floating point rounding errors వల్ల డబ్బు లెక్కల్లో తప్పులు రాకూడదు. English: Financial systems avoid `float / double` for currency due to binary rounding errors and use `BigDecimal` instead — a classic production gotcha rooted in this chapter's fundamentals.

Interview Answer

"Java has 8 primitive types that store values directly on the stack, and reference types that store a pointer to heap-allocated objects. Java is statically typed, so every variable's type is fixed at compile time."

Deep Interview Answer

"Primitives are stored by value; assigning one primitive to another copies the value. Reference types are stored by reference; assigning one reference variable to another copies the pointer, so both variables point to the same heap object. This distinction drives behavior in method parameter passing, equality checks (`==` vs `equals()`), and mutability."

Cross Questions

- Q: Is Java pass-by-value or pass-by-reference? → A: Always pass-by-value. For objects, the *value* passed is the reference (the pointer itself is copied), which is why it *looks* like pass-by-reference but isn't — reassigning the parameter inside a method never affects the caller's reference.
- Q: Why does `0.1 + 0.2 != 0.3` in Java (and most languages)? → A: IEEE-754 floating point cannot represent 0.1/0.2 exactly in binary, causing rounding errors — use `BigDecimal` for exact decimal arithmetic.
- Q: Difference between `float` and `double` default? → A: Java float literals need `f` suffix (`3.14f`) because `3.14` alone is a `double` literal by default.

Tricky Questions

- Q: What is the default value of a local `int` variable? → A: **There is none** — local variables are not auto-initialized; the compiler throws "variable might not have been initialized" if used before assignment. (Only instance/static fields get defaults.)
- Q: `byte b = 130;` — compiles? → A: No, compile error — 130 exceeds `byte` range (-128 to 127); needs an explicit cast and will overflow/wrap.

Coding Exercise

Level 1 (Beginner): Declare all 8 primitive types with sample values and print them. **Level 2 (Intermediate):** Write a program demonstrating implicit widening (`int` → `long`) and explicit narrowing cast (`double` → `int`), printing before/after values. **Level 3 (Advanced):** Demonstrate integer overflow: `int max = Integer.MAX_VALUE; System.out.println(max + 1);` — explain the output. **Level 4 (Interview):** Why is `char` in Java 2 bytes instead of 1 (as in C)? (Answer: Java uses UTF-16 internally to support Unicode.) **Level 5 (Senior):** Design a `Money` class avoiding `double` pitfalls using `BigDecimal` and `long` (cents-based) approaches — compare trade-offs. **Level 6 (Mastery):** Explain, without notes, stack vs heap storage for primitives vs references, using a diagram you draw yourself.

CHAPTER 3 — Operators

Telugu Explanation

Operators అంటే values మేడ operations perform చేసే symbols. Java లో: **Arithmetic** (+ - * / %), **Relational** (== != > < >= <=), **Logical** (&& || !), **Bitwise** (& | ^ ~ << >> >>>), **Assignment** (= += -= ...), **Ternary** (condition ? a : b), **instanceof**.

Professional English Explanation

Operators are symbols that perform operations on operands. Understanding operator precedence, short-circuit evaluation, and integer vs floating-point division is essential to avoid subtle bugs.

Java Code

```
public class OperatorsDemo {
    public static void main(String[] args) {
        int a = 10, b = 3;
        System.out.println("a / b = " + (a / b));           // integer division
        System.out.println("a % b = " + (a % b));           // modulus
        System.out.println("a / (double)b = " + (a / (double) b));

        boolean result = (a > 5) && (b < 5);                 // short-circuit AND
        System.out.println("Logical AND: " + result);

        int x = 5;
        String category = (x % 2 == 0) ? "Even" : "Odd";      // ternary
        System.out.println("Category: " + category);

        System.out.println("Bitwise AND (5 & 3): " + (5 & 3));
        System.out.println("Left shift (5 << 1): " + (5 << 1));
    }
}
```

Output

```
a / b = 3
a % b = 1
a / (double)b = 3.3333333333333335
Logical AND: true
Category: Odd
Bitwise AND (5 & 3): 1
Left shift (5 << 1): 10
```

Internal Working

- `a / b` when both are `int` performs **integer division** (truncates, doesn't round) — a top interview trap.
- `&& / ||` are **short-circuit**: the right operand is not evaluated if the left already decides the result. `& / |` on booleans always evaluate both sides (no short-circuit) — relevant when the right side has side effects.

Real-World Example

Telugu: `if (user != null && user.isActive())` లో short-circuit వల్ల `user` null అయితే `isActive()` call అవ్వదు — `NullPointerException` రాకుండా ఆగుతుంది. English: Short-circuit `&&` is the idiomatic Java null-guard pattern used everywhere in production code before calling a method on a possibly-null reference.

Interview Answer

"Java operators follow standard categories — arithmetic, relational, logical, bitwise, assignment, ternary. The two things interviewers probe most are integer division truncation and short-circuit evaluation of `&&` / `||`."

Cross Questions

- Q: Difference between `&&` and `&` on booleans? → A: `&&` short-circuits (skips right operand if left is `false`); `&` always evaluates both operands.
- Q: What does `5 / 2` return, and `5 / 2.0`? → A: `2` (integer division) vs `2.5` (one operand is `double`, promotes the whole expression).
- Q: What is operator precedence pitfall in `a + b + "text"` vs `"text" + a + b`? → A: Left-to-right evaluation — the first yields numeric addition then concatenation; the second concatenates left to right as strings from the start.

Tricky Questions

- Q: `System.out.println(1 + 2 + "3");` output? → A: `"33"` (`1+2=3` computed first, then concatenated with `"3"`).
- Q: `System.out.println("1" + 2 + 3);` output? → A: `"123"` (string concatenation left to right from the start).

Coding Exercise

L1: Print results of all arithmetic operators on two integers. **L2:** Write a program showing short-circuit `&&` skipping a method call (print inside the method to prove it wasn't called). **L3:** Implement even/odd check using bitwise `&` instead of `%`. **L4 (Interview):** Explain why `NaN == NaN` is `false` in Java. **L5 (Senior):** Explain integer overflow risk in `(a + b) / 2` for large `int` values and how to avoid it (`a + (b - a) / 2`, or use `long`). **L6 (Mastery):** Verbally derive the output of 5 chained operator-precedence expressions without running code.

CHAPTER 4 — Control Flow Statements

Telugu Explanation

Control flow అంటే program execution order ని decide చేసేది — conditions (`if/else` , `switch`) మరియు loops (`for` , `while` , `do-while`) ద్వారా.

Professional English Explanation

Control flow statements determine the order in which statements execute: conditional branching (`if-else` , `switch`) and repetition (`for` , `while` , `do-while`), plus `break` / `continue` for loop control.

Java Code

```
public class ControlFlowDemo {
    public static void main(String[] args) {
        int score = 82;

        // if-else ladder
        if (score >= 90) System.out.println("Grade A");
        else if (score >= 75) System.out.println("Grade B");
        else System.out.println("Grade C");

        // traditional switch
        int day = 3;
        switch (day) {
            case 1: System.out.println("Mon"); break;
            case 2: System.out.println("Tue"); break;
            case 3: System.out.println("Wed"); break;
            default: System.out.println("Other");
        }

        // enhanced switch (Java 14+, standard since Java 14/finalized; usable on 17/21)
        String dayName = switch (day) {
            case 1 -> "Mon";
            case 2 -> "Tue";
            case 3 -> "Wed";
            default -> "Other";
        };
        System.out.println("Enhanced switch: " + dayName);

        // loops
        for (int i = 1; i <= 3; i++) System.out.println("for i=" + i);

        int j = 0;
        while (j < 3) { System.out.println("while j=" + j); j++; }

        int k = 0;
        do { System.out.println("do-while k=" + k); k++; } while (k < 3);

        // break/continue
        for (int i = 1; i <= 5; i++) {
            if (i == 3) continue;
            if (i == 5) break;
            System.out.println("loop i=" + i);
        }
    }
}
```

Output

```
Grade B
Wed
Enhanced switch: Wed
for i=1
for i=2
for i=3
while j=0
while j=1
while j=2
do-while k=0
do-while k=1
do-while k=2
loop i=1
loop i=2
loop i=4
```

Internal Working

- Traditional `switch` **falls through** without `break` — execution continues into the next case. This is a classic bug source.
- The **enhanced switch expression** (`->`) does not fall through and can directly return a value — introduced as a preview in Java 12/13, finalized in Java 14, and used heavily in modern Java (11 codebases upgrading, 21 idiomatic code).
- `do-while` guarantees the body runs **at least once**, unlike `while`.

Real-World Example

Telugu: Payment status handle చేసేటప్పుడు `switch` వాడి `PENDING`, `SUCCESS`, `FAILED` states ని branch చేస్తారు — fall-through bug వల్ల production లో wrong notification పంపే mistakes జరుగుతాయి, అందుకే `break` critical. English: State-machine-like logic (order status, payment status) is commonly implemented with `switch`; missing `break` statements are a real production bug pattern.

Interview Answer

"Java offers `if-else` for boolean-condition branching and `switch` for discrete-value branching. Traditional `switch` falls through without `break`; the modern arrow-form `switch` expression (Java 14+) doesn't fall through and can return a value directly."

Cross Questions

- Q: Can `switch` work on `String`? → A: Yes, since Java 7.
- Q: Can `switch` work on `long` or `boolean`? → A: No — `switch` supports `byte`, `short`, `char`, `int` (and their wrappers), `String`, and `enum`; not `long` or `boolean`.
- Q: Difference between `break` and `continue`? → A: `break` exits the loop entirely; `continue` skips to the next iteration.

Tricky Questions

- Q: What happens if a `switch` case has no `break` and matches? → A: Execution "falls through" into the next case's statements regardless of its condition, until a `break` or the end of the switch.
- Q: Is `do-while(false);` valid, infinite-loop-free code? → A: Yes — it runs the body exactly once; not infinite since the condition is `false`.

Coding Exercise

L1: FizzBuzz (1 to 30) using `for` and `if/else`. **L2:** Rewrite FizzBuzz's grade logic with enhanced `switch` expressions. **L3:** Simulate a traffic light state machine using `switch` with intentional fall-through comments explaining why. **L4 (Interview):** Why was the enhanced `switch` expression introduced — what problem did fall-through cause in production code? **L5 (Senior):** Refactor a deeply nested `if-else` ladder (5+ levels) into cleaner logic using early returns / switch expressions. **L6 (Mastery):** Explain fall-through behavior and draw the control-flow diagram for a 4-case switch without `break` on case 2.

CHAPTER 5 — Arrays

Telugu Explanation

Array అంటే **fixed-size, same data type** elements ని ఒక contiguous memory block లో store చేసే data structure. Array size ఒకసారి define అయితే మారదు.

Professional English Explanation

An array is a fixed-length, homogeneous, index-based data structure stored contiguously in memory (conceptually — the JVM heap manages the actual layout). Arrays are objects in Java, so they carry a `length` field and default to `null` as a reference type.

Java Code

```
import java.util.Arrays;

public class ArraysDemo {
    public static void main(String[] args) {
        int[] numbers = {10, 20, 30, 40, 50};
        System.out.println("Length: " + numbers.length);
        System.out.println("Element at index 2: " + numbers[2]);

        for (int n : numbers) System.out.print(n + " ");
        System.out.println();

        int[][] matrix = {{1, 2}, {3, 4}, {5, 6}}; // 2D array
        for (int[] row : matrix) System.out.println(Arrays.toString(row));

        int[] copy = Arrays.copyOf(numbers, 3);
        System.out.println("Copy: " + Arrays.toString(copy));

        Arrays.sort(numbers);
        System.out.println("Sorted: " + Arrays.toString(numbers));
    }
}
```

Output

```
Length: 5
Element at index 2: 30
10 20 30 40 50
[1, 2]
[3, 4]
[5, 6]
Copy: [10, 20, 30]
Sorted: [10, 20, 30, 40, 50]
```

Internal Working / Memory View

- Arrays are **objects** stored on the **Heap**, even for primitive-type arrays. The reference variable (`numbers`) lives on the Stack, pointing to the heap-allocated array object.

- Accessing `numbers[10]` when length is 5 throws `ArrayIndexOutOfBoundsException` at **runtime** — Java performs bounds checking (unlike C/C++).

STACK HEAP
`numbers` -----> `[10][20][30][40][50]` (length=5, index 0..4)

Real-World Example

Telugu: Fixed size lookup tables (e.g., 12 months names) కి array వాడతారు; size dynamic గా మారాలంటే `ArrayList` వాడాలి (Book 05లో వివరంగా). English: Arrays are used where size is fixed and known upfront (e.g., days of week, a matrix); dynamic collections use `ArrayList / List`, covered fully in Book 05.

Interview Answer

"Arrays in Java are fixed-size, index-based, homogeneous containers implemented as objects on the heap. They provide $O(1)$ index access but $O(n)$ insertion/deletion in the middle since elements must shift."

Cross Questions

- Q: Why is array access $O(1)$? → A: The JVM computes the memory address directly as `baseAddress + index * elementSize`.
- Q: Array vs ArrayList? → A: Array is fixed-size and can hold primitives directly; `ArrayList` is resizable, holds objects (autoboxed for primitives), and offers richer API — detailed in Book 05.
- Q: What is the default value of an `int[]` array's elements vs a `String[]` array's? → A: `0` for `int[]`, `null` for `String[]` (reference type default).

Tricky Questions

- Q: `int[] arr = new int[-1];` — result? → A: Throws `NegativeArraySizeException` at runtime.
- Q: Are Java arrays covariant? → A: Yes — `Object[] objs = new String[3];` compiles, but `objs[0] = 10;` throws `ArrayStoreException` at runtime, since the actual runtime type is still `String[]`.

Coding Exercise

L1: Find max and min in an `int[]` array manually (no library methods). **L2:** Reverse an array in place without creating a new array. **L3:** Rotate an array left by `k` positions efficiently. **L4 (Interview):** Why do arrays throw `ArrayStoreException` at runtime instead of compile time for the covariance case above? **L5 (Senior):** Implement matrix multiplication for two 2D arrays with dimension validation and clear exceptions. **L6 (Mastery):** Explain array memory layout and bounds-checking behavior from memory, with a diagram.

CHAPTER 6 — Methods

Telugu Explanation

Method అనేది ఒక reusable block of code, ఒక specific task perform చేయడానికి. Parameters, return type, method signature concepts ఇక్కడ ముఖ్యం.

Professional English Explanation

A method encapsulates a reusable unit of behavior. Java supports method overloading (same name, different parameter list), pass-by-value semantics for all arguments, and variable-arity (`varargs`) parameters.

Java Code

```
public class MethodsDemo {

    static int add(int a, int b) {                // overload 1
        return a + b;
    }

    static double add(double a, double b) {        // overload 2 (overloading)
        return a + b;
    }

    static int sum(int... numbers) {               // varargs
        int total = 0;
        for (int n : numbers) total += n;
        return total;
    }

    static void modify(int x) {                    // pass-by-value proof
        x = x + 100;
    }

    public static void main(String[] args) {
        System.out.println(add(2, 3));
        System.out.println(add(2.5, 3.5));
        System.out.println(sum(1, 2, 3, 4));

        int value = 10;
        modify(value);
        System.out.println("value after modify(): " + value); // unchanged
    }
}
```

Output

```
5
6.0
10
value after modify(): 10
```

Internal Working / Memory View

- Each method call creates a new **stack frame** on the call stack, holding local variables and parameters; the frame is popped when the method returns.
- `modify(value)` copies `10` into parameter `x`; changes to `x` never affect the caller's `value` — Java is **always pass-by-value**, even for object references (the reference *value* is copied).

Real-World Example

Telugu: Service layer methods (e.g., `calculateDiscount(Order order)`) overloading వాడి different input types (percentage discount vs flat discount) handle చేస్తారు. English: Overloading is used extensively in service/utility classes — e.g., multiple `log()` overloads for different argument combinations, or `String.valueOf()` overloads in the JDK itself.

Interview Answer

"A method is a named, reusable block of code with a defined signature (name + parameter types). Java supports overloading based on parameter list differences, and all arguments are passed by value — including object references, where the reference's value (the pointer) is copied, not the object itself."

Deep Interview Answer

"Overload resolution happens at compile time based on the static (declared) type of arguments — this is why overloading is a form of compile-time polymorphism, distinct from overriding, which resolves at runtime based on the actual object type (Book 02)."

Cross Questions

- Q: Can two methods differ only by return type? → A: No — return type alone doesn't make a valid overload; the compiler needs a distinguishable parameter list.
- Q: What is method signature exactly? → A: Method name + parameter types (in order) — NOT the return type or parameter names.
- Q: Can varargs coexist with regular parameters? → A: Yes, but varargs must be the last parameter.

Tricky Questions

- Q: If you pass a mutable object (e.g., an array or a custom object) to a method and mutate its *internal state* (not reassign the reference), does the caller see the change? → A: Yes — because the copied reference still points to the same heap object; mutating that object's fields is visible to the caller. Reassigning the parameter itself is not.
- Q: `add(2, 3.0)` — which overload from the demo above is called? → A: `add(double, double)`, because `2` widens to `2.0` (implicit widening conversion) to match.

Coding Exercise

L1: Write 3 overloaded `area()` methods for square, rectangle, circle. **L2:** Write a varargs method `max(int... nums)` returning the largest value. **L3:** Demonstrate the "mutate object vs reassign reference" distinction with a custom class and a method. **L4 (Interview):** Explain why Java doesn't support default parameter values like some other languages (and how overloading/varargs fill that gap). **L5 (Senior):** Design a small `Logger` utility class with multiple overloaded `log()` methods handling different signatures cleanly (no ambiguity). **L6 (Mastery):** Explain stack frames and pass-by-value from memory using a diagram, including the object-reference nuance.

CHAPTER 7 — Classes & Objects (Foundations)

Telugu Explanation

Class అనేది ఒక **blueprint/template** — object ఎలా ఉండాలి (fields) మరియు ఏమి చేయగలదో (methods) define చేస్తుంది. Object అనేది ఆ blueprint ఆధారంగా **runtime** లో create అయిన actual entity, heap మీద store అవుతుంది.

Professional English Explanation

A class defines the structure (fields) and behavior (methods) of a type. An object is a runtime instance of a class, created via the `new` keyword, allocated on the heap, and referenced via a variable on the stack.

Java Code

```
class Student {
    String name;
    int age;

    void introduce() {
        System.out.println("Hi, I am " + name + ", age " + age);
    }
}

public class ClassesObjectsDemo {
    public static void main(String[] args) {
        Student s1 = new Student();
        s1.name = "Anitha";
        s1.age = 21;

        Student s2 = new Student();
        s2.name = "Kiran";
        s2.age = 23;

        s1.introduce();
        s2.introduce();

        System.out.println("s1 == s2 ? " + (s1 == s2));           // reference comparison
    }
}
```

Output

```
Hi, I am Anitha, age 21
Hi, I am Kiran, age 23
s1 == s2 ? false
```

Internal Working / Memory View

```
STACK          HEAP
s1 -----> [Student obj#1: name="Anitha", age=21]
s2 -----> [Student obj#2: name="Kiran", age=23]
```

- `new Student()` triggers: (1) memory allocation on the heap, (2) field defaults applied (`null` , `0`), (3) constructor execution, (4) reference returned and assigned to `s1` .
- Class metadata (method bytecode, static fields) lives in the **Method Area / Metaspace** (Book 03), not duplicated per object — each object only stores its own instance field values.

Real-World Example

Telugu: `Student` class లానే production లో `Order` , `Customer` , `Product` లాంటి domain classes design చేస్తారు — ఇవే entity classes JPA (Book 13) లో persist అవుతాయి. English: This is exactly how domain/entity classes are modeled in real backend systems — the same `class` mechanics here scale directly into JPA `@Entity` classes later in the series.

Interview Answer

"A class is a blueprint defining fields and methods; an object is a runtime instance of that blueprint, allocated on the heap. Multiple objects of the same class share the same method code (from the Method Area) but each maintains its own copy of instance field values."

Cross Questions

- Q: Where do instance variables live vs static variables? → A: Instance variables live inside each object on the heap; static variables live once per class in the Method Area/Metaspace, shared across all instances.
- Q: Why did `s1 == s2` print `false` even though both have similar-looking data conceptually? → A: `==` compares references (memory addresses) for objects, not field content — they are two distinct heap objects.
- Q: What are default field values before constructor runs? → A: `null` for references, `0 / 0.0` for numeric primitives, `false` for boolean — unlike local variables, fields *are* auto-initialized.

Tricky Questions

- Q: Can a class exist without ever being instantiated? → A: Yes — e.g., a utility class with only `static` methods (like `Math`) is never instantiated.
- Q: Is `null` a keyword or a value in Java? → A: A literal reference value (not an object, not a keyword like `class`), representing "no object".

Coding Exercise

L1: Create a `Car` class with fields `brand` , `model` , `year` and a `display()` method; instantiate 2 objects. **L2:** Add a method that compares two `Car` objects field-by-field for equality (manual, not `equals()` yet — that's Chapter 15). **L3:** Create an array of 5 `Student` objects and print all names using a loop. **L4 (Interview):** Explain the difference between a class and an object using a real-

world analogy plus the technical memory model. **L5 (Senior):** Model a small domain (`Order` has a `Customer` and a `List`-like array of `Product`) purely with classes/objects/arrays (no collections yet). **L6 (Mastery):** Draw the heap/stack diagram for 3 objects of the same class from memory.

CHAPTER 8 — Constructors & `this`

Telugu Explanation

Constructor అనేది object create అయినప్పుడు automatically call అయ్యే special method, object ని initialize చేయడానికి. దీని పేరు class పేరు తోనే ఉంటుంది, return type ఉండదు. `this` keyword current object ని refer చేస్తుంది — parameter name, field name same అయినప్పుడు differentiate చేయడానికి వాడతారు.

Professional English Explanation

A constructor initializes a newly created object; it shares the class name and has no return type. `this` refers to the current instance and is used to disambiguate fields from parameters, and to chain constructors (`this(...)`) within the same class.

Java Code

```
class Employee {
    String name;
    double salary;

    Employee() {                // no-arg constructor
        this("Unnamed", 0.0);   // constructor chaining
    }

    Employee(String name, double salary) { // parameterized constructor
        this.name = name;        // 'this' disambiguates field vs parameter
        this.salary = salary;
    }

    void show() {
        System.out.println(name + " earns " + salary);
    }
}

public class ConstructorDemo {
    public static void main(String[] args) {
        Employee e1 = new Employee();
        Employee e2 = new Employee("Suresh", 55000.0);
        e1.show();
        e2.show();
    }
}
```

Output

```
Unnamed earns 0.0
Suresh earns 55000.0
```

Internal Working

- If a class defines **no constructor**, the compiler inserts a **default no-arg constructor** automatically. The moment you define **any** constructor yourself, that automatic default

disappears.

- `this(...)` **must be the first statement** in a constructor — this enforces a single initialization chain and avoids duplicate/partial init logic.
- Constructor chaining ensures shared initialization logic lives in one place (DRY principle, formalized in Book 02/19).

Real-World Example

Telugu: JPA `@Entity` classes లో no-arg constructor **mandatory** — framework reflection ద్వారా object create చేయడానికి దీన్ని వాడుతుంది (Book 13 లో వివరంగా). English: Frameworks like Hibernate/JPA require a no-arg constructor because they instantiate entities via reflection before populating fields — a direct real-world consequence of this chapter.

Interview Answer

"A constructor initializes an object at creation time, shares the class name, and has no return type. `this` refers to the current object — commonly used to resolve field/parameter naming conflicts and to chain constructors within the same class via `this(...)`."

Deep Interview Answer

"Constructor chaining via `this(...)` centralizes initialization logic to avoid duplication and partial-initialization bugs. It differs from `super(...)` (Book 02), which chains to the parent class's constructor instead of a sibling constructor in the same class — and `this(...)` / `super(...)` cannot both appear in the same constructor since both must be the first statement."

Cross Questions

- Q: Can a constructor be `private`? → A: Yes — used in Singleton pattern (Book 18) and static factory method patterns to control instantiation.
- Q: Can a constructor be `final`, `static`, or `abstract`? → A: No to all three — constructors aren't inherited or overridden, so those modifiers are meaningless on them.
- Q: What happens if you call `this()` not as the first statement? → A: Compile-time error.

Tricky Questions

- Q: If a class has only a parameterized constructor, is `new MyClass()` still valid? → A: No — compile error, since defining any constructor removes the compiler-generated default no-arg one.
- Q: Can constructor chaining be circular (`A() -> B() -> A()`)? → A: No, the compiler detects and rejects recursive constructor chains.

Coding Exercise

L1: Add 2 more overloaded constructors to `Employee` (e.g., name-only, salary-only) using `this(...)` chaining. **L2:** Write a `Point` class with fields `x`, `y`, using `this.x` / `this.y` to resolve naming conflicts. **L3:** Implement a private constructor + static factory method (`createDefault()`)

pattern. **L4 (Interview):** Why must `this()` / `super()` be the first line of a constructor? **L5 (Senior):** Explain why frameworks like Hibernate require a no-arg constructor on entities, connecting it to reflection-based instantiation. **L6 (Mastery):** Explain constructor chaining and the compiler-generated default constructor rule from memory.

CHAPTER 9 — static and final

Telugu Explanation

`static` అంటే — ఆ member (variable/method) **class కి సంబంధించినది**, ఏ specific object కి కాదు. అందరు objects ఒకే copy ని share చేస్తారు. `final` అంటే — ఒకసారి assign చేస్తే మార్పులేము (variable కి), override చేయలేము (method కి), extend చేయలేము (class కి).

Professional English Explanation

`static` members belong to the class itself, shared across all instances, and accessible without object creation. `final` enforces immutability/non-modification: a `final` variable can be assigned exactly once, a `final` method cannot be overridden, and a `final` class cannot be extended.

Java Code

```
class Counter {
    static int totalCount = 0;        // shared across all instances
    final String id;                  // must be set once (constructor or declaration)

    Counter() {
        totalCount++;
        this.id = "ID-" + totalCount;
    }

    static void printTotal() {        // static method: no 'this', no instance state access
        System.out.println("Total counters created: " + totalCount);
    }
}

final class ImmutablePoint {        // final class: cannot be extended
    final int x, y;
    ImmutablePoint(int x, int y) { this.x = x; this.y = y; }
}

public class StaticFinalDemo {
    public static void main(String[] args) {
        new Counter();
        new Counter();
        Counter c3 = new Counter();

        Counter.printTotal();        // called via class name, idiomatic
        System.out.println(c3.id);

        ImmutablePoint p = new ImmutablePoint(3, 4);
        System.out.println(p.x + ", " + p.y);
        // p.x = 10; // compile error - final field
    }
}
```

Output

```
Total counters created: 3
ID-3
3, 4
```

Internal Working / Memory View

- `static` fields live **once per class** in the Method Area/Metaspace, not per object — all `Counter` instances read/write the same `totalCount` slot.
- `static` methods cannot use `this` (no implicit instance) and cannot access non-static (instance) members directly.
- `final` on a primitive locks the value; `final` on a reference locks the *reference itself* (can't reassign to a different object) — but the referenced object's internal state can still change if it's mutable (a very common interview trap).

METHOD AREA (once per class)	HEAP (per object)
<code>Counter.totalCount = 3</code>	<code>obj1{id="ID-1"}</code>
	<code>obj2{id="ID-2"}</code>
	<code>obj3{id="ID-3"}</code>

Real-World Example

Telugu: `public static final double TAX_RATE = 0.18;` లాంటి constants production code లో everywhere వాడతారు — value మారదు, class instance అవసరం లేకుండా access చేయచ్చు. English: `static final` constants are the standard way to define application-wide constants (tax rates, config keys); `static` counters/caches are used carefully (thread-safety caveats come in Book 08).

Interview Answer

"`static` members belong to the class and are shared across all instances, accessible without object creation. `final` prevents reassignment of a variable, overriding of a method, or subclassing of a class — it is Java's core mechanism for enforcing immutability and design intent."

Deep Interview Answer

"A `final` reference variable only freezes the reference binding, not the referenced object's mutable internal state — `final List list = new ArrayList<>(); list.add(x);` is legal, but `list = new ArrayList<>();` is not. True immutability requires the object's own fields to also be `final` and unexposed (Book 02/07 cover immutable design fully). `static` state is a common source of shared-mutable-state bugs in multithreaded code (Book 08)."

Cross Questions

- Q: Can a `static` method be overridden? → A: No — it can be **hidden** (redeclared in a subclass), resolved by the reference's compile-time type, not runtime polymorphism. True overriding needs instance methods (Book 02).
- Q: Can an interface have `static` methods? → A: Yes, since Java 8 (Book 07).

- Q: Why can't `static` methods access instance variables directly? → A: Because there is no implicit `this` — a static method can run without any object existing at all.

Tricky Questions

- Q: Is a `final` object's state always immutable? → A: No — `final` only fixes the *reference*; the object itself is immutable only if its class is designed that way (all fields `final`, no setters, defensive copies).
- Q: Can a `static` block run before `main()`? → A: Yes — static initializer blocks run once, in order, when the class is first loaded, before `main()` executes.

Coding Exercise

L1: Build a `Counter` class tracking total instances created (like the example) and print the count after creating 5 objects. **L2:** Demonstrate the "final reference, mutable content" trap using a `final StringBuilder` or array. **L3:** Write a class with a `static` initializer block that logs "Class loaded" and observe when it runs relative to `main()`. **L4 (Interview):** Why is `public static void main` declared exactly that way — explain each keyword's necessity. **L5 (Senior):** Identify and fix a shared-mutable-`static`-state bug in a small multi-object simulation (conceptual precursor to Book 08's thread-safety topics). **L6 (Mastery):** Explain, without notes, where `static` and instance data physically live in memory and why `static` methods can't see instance fields.

CHAPTER 10 — Access Modifiers & Packages

Telugu Explanation

Access modifiers అంటే class, method, field లకు **visibility/accessibility** control చేసేవి: `public`, `protected`, default (no modifier / package-private), `private`. Package అనేది related classes ని group చేసే namespace — folder structure తో సరిపోతుంది.

Professional English Explanation

Access modifiers control visibility scope. Packages provide namespacing and organize related classes, also forming the basis of encapsulation at the module/library level.

Access Modifier Table

Modifier	Same Class	Same Package	Subclass (diff. package)	Everywhere
<code>private</code>	✓	✗	✗	✗
default (no modifier)	✓	✓	✗	✗
<code>protected</code>	✓	✓	✓	✗
<code>public</code>	✓	✓	✓	✓

Java Code

```
package com.example.bank;

public class Account {
    private double balance; // hidden outside class - encapsulation
    protected String accountType; // visible to subclasses/same package
    String branch; // default/package-private
    public String accountHolder; // fully public (generally discouraged for fields)

    public Account(String accountHolder, double balance) {
        this.accountHolder = accountHolder;
        this.balance = balance;
    }

    public double getBalance() { // public getter – controlled access
        return balance;
    }

    private void auditLog(String msg) { // internal-only helper
        System.out.println("[AUDIT] " + msg);
    }

    public void deposit(double amount) {
        balance += amount;
        auditLog("Deposited " + amount);
    }
}
```

```
import com.example.bank.Account;

public class AccessModifiersDemo {
    public static void main(String[] args) {
        Account acc = new Account("Meena", 1000.0);
        acc.deposit(500.0);
        System.out.println("Balance: " + acc.getBalance());
        // acc.balance -> compile error, private
        // acc.auditLog("x") -> compile error, private
    }
}
```

Output

```
[AUDIT] Deposited 500.0
Balance: 1500.0
```

Internal Working

- Access checks are enforced by the **compiler** (compile-time) and re-verified by the **bytecode verifier** at class-loading time — so access control cannot be bypassed just by editing bytecode naively.
- Packages map to directory structure on disk (`com/example/bank/Account.java`) and become part of the fully-qualified class name (`com.example.bank.Account`).

Real-World Example

Telugu: `balance` ని `private` గా పెట్టి, `getBalance()` / `deposit()` ద్వారానే access ఇవ్వడం — ఇది **encapsulation** యొక్క foundation, direct field manipulation వల్ల invalid state రాకుండా ఆపుతుంది.

English: This `Account` example is the canonical encapsulation pattern used in every real domain/entity class — direct field access is blocked so all state changes go through validated methods.

Interview Answer

"Java has four access levels — `private`, default (package-private), `protected`, and `public` — controlling visibility from most to least restrictive. Encapsulation is achieved by making fields `private` and exposing controlled access via `public` methods."

Cross Questions

- Q: What is the default access modifier for a top-level class? → A: Package-private (accessible only within the same package) if no modifier is specified.
- Q: Can a top-level class be `private` or `protected`? → A: No — only `public` or default (package-private) are allowed for top-level classes; `private` / `protected` are valid for members and nested classes.
- Q: Why prefer `private` fields with public getters/setters over public fields? → A: Encapsulation — you can add validation, logging, or change internal representation later without breaking external code.

Tricky Questions

- Q: Can a `protected` member be accessed from an unrelated class in a different package? → A: No — only from the same package, or from a subclass (even in a different package), and even then only via inheritance-relevant access, not through an arbitrary instance reference in some cases (a nuance fully explored in Book 02).
- Q: Does package-private access consider subclasses in other packages? → A: No — package-private is purely about package location, ignoring inheritance across package boundaries.

Coding Exercise

L1: Convert a class with all-public fields into a properly encapsulated class with `private` fields + getters/setters. **L2:** Create two packages (`com.example.a` , `com.example.b`) and demonstrate what is/isn't accessible across them for each modifier. **L3:** Build a `BankAccount` class enforcing that `withdraw()` validates sufficient balance before mutating state (private field + public validated method). **L4 (Interview):** Why does Java favor "fields private, behavior public" as an OOP default? **L5 (Senior):** Design a small internal utility package where only specific classes are exposed `public` and helper classes stay package-private — explain the API-surface reasoning. **L6 (Mastery):** Recreate the access-modifier table from memory with an example scenario for each cell.

CHAPTER 11 — String, StringBuilder, StringBuffer

Telugu Explanation

`String` **immutable** — ఒకసారి create అయితే content మారదు; ఏదైనా operation (concatenation, replace) కొత్త `String` object create చేస్తుంది. `StringBuilder` **mutable**, fast, కానీ **thread-unsafe**. `StringBuffer` **mutable మరియు thread-safe** (synchronized), కానీ అందుకే కొంచెం slow.

Professional English Explanation

`String` is immutable — every modification creates a new object. `StringBuilder` is a mutable, high-performance, non-thread-safe character sequence builder. `StringBuffer` is functionally identical to `StringBuilder` but synchronized (thread-safe), at a performance cost.

Java Code

```
public class StringDemo {
    public static void main(String[] args) {
        String s1 = "Java";
        String s2 = "Java";
        String s3 = new String("Java");

        System.out.println(s1 == s2);           // true - same String Pool reference
        System.out.println(s1 == s3);           // false - s3 is a new heap object
        System.out.println(s1.equals(s3));       // true - content comparison

        String result = "";
        for (int i = 0; i < 3; i++) {
            result += i;                         // creates a NEW String object each time
        }                                        (inefficient)
        System.out.println(result);

        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < 3; i++) {
            sb.append(i);                        // mutates the same buffer (efficient)
        }
        System.out.println(sb.toString());

        StringBuffer sbf = new StringBuffer("Thread-safe");
        sbf.append("!");
        System.out.println(sbf);
    }
}
```

Output

```
true
false
true
012
012
Thread-safe!
```


Internal Working / Memory View — String Pool

```
HEAP
+-----+
| String Constant Pool |
| "Java" <---- s1      |
|      <---- s2 (reused) |
+-----+
| Regular Heap         |
| String object "Java" <-- s3 |
+-----+
```

- String literals are **interned** — the JVM reuses one pooled object for identical literal content, so `s1 == s2` is `true`.
- `new String("Java")` **forces** a new object outside the pool guarantee, so `s1 == s3` is `false` even though content matches.
- Each `+=` in a loop creates a new `String` object and discards the old one — **$O(n^2)$** behavior for `n` concatenations, which is why `StringBuilder` ($O(n)$ amortized) is the production-correct choice in loops.

Real-World Example

Telugu: Log messages build చేసేటప్పుడు loop లో `String +=` వాడితే performance degrade అవుతుంది పెద్ద data కి — production code `StringBuilder` వాడాలి. Multi-threaded shared buffer అయితే `StringBuffer` లేదా better, thread-confined `StringBuilder` వాడాలి. English: Building large strings (reports, logs, JSON manually) in a loop must use `StringBuilder`; `StringBuffer` is reserved for the rare case of genuinely sharing one mutable buffer across threads (usually avoided in favor of better designs — Book 08).

Interview Answer

"`String` is immutable, so every modification allocates a new object; `StringBuilder` is a mutable, non-synchronized buffer ideal for loops/heavy concatenation; `StringBuffer` is the same as `StringBuilder` but synchronized, trading performance for thread safety."

Deep Interview Answer

"String immutability enables safe sharing (no defensive copies needed), safe use as `HashMap` keys (hashcode can be cached since content never changes), and string pooling for memory efficiency. The trade-off is allocation churn during heavy mutation, which `StringBuilder` / `StringBuffer` solve by using an internal resizable `char[]` / `byte[]` buffer that's mutated in place."

Cross Questions

- Q: Why is `String` immutable by design? → A: Security (safe as method arguments — can't be altered by callee), thread-safety (safely shared without synchronization), hashcode caching (safe as `HashMap` / `HashSet` keys), and string pool reuse.
- Q: Does `StringBuilder` guarantee any particular internal growth strategy? → A: It doubles capacity (roughly) when the buffer is full, amortizing the cost of growth across appends.

- Q: When would you still choose `StringBuffer` over `StringBuilder`? → A: Only when multiple threads must literally mutate the *same* buffer concurrently and you want the JDK's built-in synchronization rather than rolling your own — rare in modern designs that prefer immutable message passing.

Tricky Questions

- Q: `String a = "Hi"; a.concat(" there"); System.out.println(a);` — output? → A: `"Hi"` — `concat` returns a *new* `String`; since it's not assigned back, `a` is unchanged (a classic immutability trap).
- Q: How many objects are created by `String s = new String("test");`? → A: Up to 2 — one in the pool (if `"test"` literal wasn't already pooled) and one new object on the regular heap via `new`.

Coding Exercise

L1: Demonstrate `==` vs `.equals()` for 4 different `String` creation styles (literal, `new`, `.intern()`, concatenation). **L2:** Benchmark (using `System.nanoTime()`) 10,000 concatenations via `+=` vs `StringBuilder.append()` and print the time difference. **L3:** Write a palindrome checker using `StringBuilder.reverse()`. **L4 (Interview):** Why does `HashMap` benefit specifically from `String` being immutable? **L5 (Senior):** Design a log-line builder utility that safely handles concurrent calls — decide between `StringBuffer`, thread-local `StringBuilder`, or an immutable-message approach, and justify the choice. **L6 (Mastery):** Explain `String` Pool, immutability rationale, and `StringBuilder` internal growth from memory with a diagram.

CHAPTER 12 — Wrapper Classes, Autoboxing & Unboxing

Telugu Explanation

Wrapper classes అంటే primitive types ని objects గా represent చేసేవి: `int` → `Integer` , `double` → `Double` , `char` → `Character` , `boolean` → `Boolean` etc. **Autoboxing** అంటే primitive ని automatic గా wrapper object గా convert చేయడం; **Unboxing** అంటే దానికి వ్యతిరేకం.

Professional English Explanation

Wrapper classes provide object representations of primitives, necessary anywhere Java requires an object (Collections, generics — Book 05/06). Autoboxing/unboxing is compiler-inserted automatic conversion between primitives and their wrapper types, introduced in Java 5.

Java Code

```
import java.util.ArrayList;
import java.util.List;

public class WrapperDemo {
    public static void main(String[] args) {
        int primitive = 10;
        Integer boxed = primitive;           // autoboxing: int -> Integer
        int unboxed = boxed;                  // unboxing: Integer -> int

        List<Integer> numbers = new ArrayList<>();
        numbers.add(5);                       // autoboxed: int 5 -> Integer
        int first = numbers.get(0);           // unboxed: Integer -> int

        Integer a = 127, b = 127;
        System.out.println("127 == 127 (cached): " + (a == b)); // true - Integer cache

        Integer c = 200, d = 200;
        System.out.println("200 == 200 (not cached): " + (c == d)); // false - outside cache
range

        Integer nullInt = null;
        try {
            int crash = nullInt;               // NPE on unboxing null
        } catch (NullPointerException e) {
            System.out.println("Caught NPE while unboxing null Integer");
        }
    }
}
```

Output

```
127 == 127 (cached): true
200 == 200 (not cached): false
Caught NPE while unboxing null Integer
```

Internal Working

- The compiler rewrites `Integer boxed = primitive;` into `Integer boxed = Integer.valueOf(primitive);`, and unboxing into `.intValue()` calls — this is purely **syntactic sugar**, not a JVM-level feature.
- `Integer.valueOf(int)` **caches** instances for values **-128 to 127** (the "Integer Cache"), so autoboxed values in that range are reference-equal (`== true`); outside that range, new objects are created each time.
- Unboxing a `null` wrapper triggers an implicit `.intValue()` call on `null`, throwing `NullPointerException` — a very common production bug (e.g., a `Map` returning `null` unboxed into a primitive field).

Real-World Example

Telugu: DB నుండి nullable column ఒక `Integer` గా వస్తే, direct గా `int` field కి assign చేస్తే `null` వచ్చినప్పుడు NPE వస్తుంది — ఇది real production bug pattern. English: A nullable database column mapped to a wrapper type, then carelessly unboxed into a primitive, is one of the most common real-world NPE sources — always check for `null` before unboxing, or keep the field as the wrapper type when `null` is meaningful.

Interview Answer

"Wrapper classes give object representations of primitives, required by Collections and generics which only work with objects. Autoboxing/unboxing is automatic compiler-inserted conversion between primitives and wrappers, but it can silently throw `NullPointerException` when unboxing a `null` wrapper."

Cross Questions

- Q: Why does `Integer a = 127, b = 127; a == b` return `true` but `200/200` return `false`? → A: The JVM caches `Integer` objects for -128..127 via `Integer.valueOf()`; values outside that range always allocate new objects.
- Q: Should you use `==` or `.equals()` to compare wrapper objects? → A: Always `.equals()` for value comparison — `==` is unreliable due to caching behavior.
- Q: Does every wrapper type have caching? → A: `Byte`, `Short`, `Integer`, `Long` cache -128..127; `Character` caches 0..127; `Boolean` caches `TRUE / FALSE`; `Float / Double` are **never** cached (they don't override caching since floating point equality/identity is less meaningful this way).

Tricky Questions

- Q: `Long l = 128L; long p = 128; System.out.println(l == p);` — output? → A: `true` — when comparing a wrapper to a primitive, the wrapper is unboxed first, so it becomes a primitive comparison, bypassing the cache issue entirely.
- Q: Autoboxing in a loop — what's the hidden cost? → A: Each iteration can allocate a new wrapper object outside the cache range, creating unnecessary garbage — a performance smell in hot loops.

Coding Exercise

L1: Print the result of `==` comparison for cached (100) and uncached (1000) `Integer` values. **L2:** Reproduce the null-unboxing `NullPointerException` deliberately, then fix it with a null check. **L3:** Write a method that safely sums a `List<Integer>` that may contain `null` elements without crashing. **L4 (Interview):** Explain why `Integer` caching exists and its range. **L5 (Senior):** Review a hypothetical DAO method that unboxes a nullable DB column directly into a primitive `int` field — identify the bug and fix the design. **L6 (Mastery):** Explain autoboxing/unboxing as compiler sugar and the Integer Cache mechanism from memory.

CHAPTER 13 — Enums

Telugu Explanation

Enum అంటే fixed set of constants ని represent చేసే special type — ఉదాహరణకి రోజులు, status values (PENDING , SUCCESS , FAILED). Enums type-safe గా ఉంటాయి — invalid value assign చేయలేరు, ఇది plain int / String constants కంటే safer.

Professional English Explanation

An `enum` defines a fixed, type-safe set of named constants. Java enums are full classes internally — they can have fields, constructors, methods, and even abstract methods with per-constant implementations.

Java Code

```
enum OrderStatus {
    PENDING("Order placed, awaiting confirmation"),
    SHIPPED("Order has been shipped"),
    DELIVERED("Order delivered successfully"),
    CANCELLED("Order was cancelled");

    private final String description;

    OrderStatus(String description) { // enum constructor - always private implicitly
        this.description = description;
    }

    String getDescription() {
        return description;
    }
}

public class EnumDemo {
    public static void main(String[] args) {
        OrderStatus status = OrderStatus.SHIPPED;

        System.out.println(status); // SHIPPED (toString default = name)
        System.out.println(status.getDescription());
        System.out.println(status.ordinal()); // position: 0-based
        System.out.println(status.name());

        switch (status) {
            case PENDING -> System.out.println("Waiting...");
            case SHIPPED -> System.out.println("On the way!");
            case DELIVERED -> System.out.println("Done!");
            case CANCELLED -> System.out.println("Cancelled.");
        }

        for (OrderStatus s : OrderStatus.values()) {
            System.out.println(s.name() + " -> " + s.getDescription());
        }
    }
}
```

Output

```
SHIPPED
Order has been shipped
1
SHIPPED
On the way!
PENDING -> Order placed, awaiting confirmation
SHIPPED -> Order has been shipped
DELIVERED -> Order delivered successfully
CANCELLED -> Order was cancelled
```

Internal Working

- Each enum constant is actually a `public static final` instance of the enum type, created once when the enum class loads.
- Every enum implicitly extends `java.lang.Enum` — this is why enums **cannot extend any other class** (Java has single inheritance for classes) but **can implement interfaces**.
- `switch` on enums is exhaustive-friendly with modern arrow-style syntax and doesn't need the enum's class-qualified name in case labels.

Real-World Example

Telugu: Payment status, HTTP method type (`GET` , `POST` ...), day of week — ఇవన్నీ enum నా model చేస్తే, typo-based bugs ("pendign" లాంటి string typos) compile-time లోనే catch అవుతాయి. English: Enums are the standard way to model a fixed domain of states (order status, payment status, roles) — replacing error-prone `String` / `int` constants with compile-time-checked, self-documenting types.

Interview Answer

"An enum is a special class representing a fixed set of constants. Java enums are full-featured classes — they can have constructors, fields, and methods — and each constant is a singleton instance of the enum type created at class-loading time."

Deep Interview Answer

"Enums implicitly extend `java.lang.Enum<E>` , which is why multiple inheritance of classes isn't possible with enums, though implementing interfaces is fully supported — enabling patterns like strategy-per-constant, where each constant overrides an abstract method."

Cross Questions

- Q: Can an enum implement an interface? → A: Yes.
- Q: Can an enum have an abstract method with different implementations per constant? → A: Yes — each constant can override the abstract method in its own constant body.
- Q: Are enum constructors ever `public` ? → A: No — they are implicitly `private` (or package-private); constants are the only instances, created internally by the JVM.

Tricky Questions

- Q: Can you use `==` to compare enum constants safely? → A: Yes — since each constant is a singleton, `==` is both safe and idiomatic (unlike Strings), and is what `switch` relies on internally.
- Q: What does `values()` return, and is it cheap to call repeatedly in a hot loop? → A: A **new array copy** every time it's called — calling it repeatedly in performance-critical code is wasteful; cache the array if needed.

Coding Exercise

L1: Create a `Day` enum (`MONDAY ... SUNDAY`) with a method `isWeekend()` . **L2:** Extend the `OrderStatus` enum with an abstract method `nextStatus()` implemented differently per constant. **L3:** Use `EnumMap` -style thinking (conceptually) to map each `OrderStatus` to a notification message using a `switch` expression. **L4 (Interview):** Why are enums preferred over `int / String` constants for representing fixed domains? **L5 (Senior):** Model a small state machine (`OrderStatus` transitions) using enum + abstract method, rejecting invalid transitions. **L6 (Mastery):** Explain why enums can't extend classes but can implement interfaces, from memory.

CHAPTER 14 — Nested, Inner & Anonymous Classes

Telugu Explanation

Nested class అంటే ఒక class లోపల define అయిన మరో class. Types: **Static nested class** (outer instance అవసరం లేదు), **Inner class** (non-static, outer instance కి attached), **Local class** (method లోపల define), **Anonymous class** (name లేని, ఒకసారి వాడే inline class).

Professional English Explanation

Java supports four nested-type forms: static nested classes (independent of an outer instance), (non-static) inner classes (implicitly hold a reference to an enclosing instance), local classes (defined inside a method body), and anonymous classes (unnamed, single-use, typically implementing an interface or extending a class inline).

Java Code

```
class Outer {
    private int outerField = 100;

    static class StaticNested {                // does not need Outer instance
        void show() { System.out.println("Static nested class"); }
    }

    class Inner {                             // needs Outer instance implicitly
        void show() { System.out.println("Inner class sees outerField=" + outerField); }
    }

    void localClassDemo() {
        class LocalHelper {                  // local class - method-scoped
            void help() { System.out.println("Local class helper"); }
        }
        new LocalHelper().help();
    }
}

interface Greeting {
    String greet(String name);
}

public class NestedClassesDemo {
    public static void main(String[] args) {
        Outer.StaticNested sn = new Outer.StaticNested();
        sn.show();

        Outer outer = new Outer();
        Outer.Inner inner = outer.new Inner();    // note the syntax: needs outer instance
        inner.show();

        outer.localClassDemo();

        Greeting anon = new Greeting() {        // anonymous class implementing interface
            @Override
            public String greet(String name) {
                return "Hello, " + name + "!";
            }
        };
        System.out.println(anon.greet("Priya"));
    }
}
```

Output

```
Static nested class
Inner class sees outerField=100
Local class helper
Hello, Priya!
```

Internal Working

- A non-static **inner class** secretly holds a hidden reference to its enclosing instance (`Outer.this`), which is why it needs `outer.new Inner()` syntax and can access `outer`'s private fields directly.
- A **static nested class** holds no such reference — it behaves like a normal top-level class that's simply namespaced inside `Outer` for organizational purposes.
- **Anonymous classes** compile to a separate `.class` file named like `NestedClassesDemo$1.class` — the compiler auto-generates the class.
- Since Java 8, anonymous classes implementing a single-abstract-method interface are frequently replaced with **lambda expressions** (Book 07) — much less boilerplate for the same effect.

Real-World Example

Telugu: GUI event handlers, old-style `Runnable` / `Comparator` implementations లో anonymous classes విస్తృతంగా వాడేవారు — ఇప్పుడు Java 8+ లో వీటిని lambdas replace చేశాయి. English: Anonymous classes were the pre-Java-8 idiom for one-off interface implementations (`Runnable`, `Comparator`, event listeners); Book 07 shows how lambdas replace most of these use cases with far less boilerplate.

Interview Answer

"Java supports static nested classes (independent of an outer instance), inner classes (implicitly tied to an enclosing instance), local classes (method-scoped), and anonymous classes (unnamed, single-use implementations) — each suited to different scoping and lifecycle needs."

Cross Questions

- Q: Why does creating an inner class need `outer.new Inner()`? → A: Because an inner class instance is conceptually bound to a specific outer instance — the JVM needs that instance to resolve the hidden enclosing reference.
- Q: Can a static nested class access the outer class's instance fields directly? → A: No — it has no implicit reference to any outer instance, only to `static` members of the outer class.
- Q: What replaced most anonymous class usage in modern Java? → A: Lambda expressions and method references (Java 8+, Book 07), for functional-interface cases.

Tricky Questions

- Q: Can a local class access local variables of the enclosing method? → A: Yes, but only if those variables are effectively final (never reassigned after initialization) — the compiler captures them by value.
- Q: Does an anonymous class have a name at all? → A: Not a name you can reference directly in source, but the compiler assigns it an internal name like `Outer$1` in bytecode.

Coding Exercise

L1: Convert the `Greeting` anonymous class example into a lambda expression (preview — full detail in Book 07). **L2:** Write a static nested class `Builder` inside a `Pizza` class to demonstrate the Builder pattern shape (full pattern detail in Book 18). **L3:** Demonstrate the "effectively final" capture rule by writing a local class that reads an enclosing method's local variable. **L4 (Interview):** What's the practical difference between a static nested class and a top-level class in a separate file? **L5 (Senior):** Decide, for a small event-driven simulation, whether each listener should be a named class, inner class, or anonymous class — justify each choice. **L6 (Mastery):** Explain, from memory, the hidden-reference difference between static nested and (non-static) inner classes.

CHAPTER 15 — Object Class Basics: `equals()`, `hashCode()`, `toString()`, Immutability Intro

Telugu Explanation

ప్రతి Java class implicit గా `Object` class ని extend చేస్తుంది. `Object` ఇచ్చే ముఖ్యమైన methods: `equals()` (content-based equality కోసం override చేయాలి), `hashCode()` (hash-based collections కోసం, `equals()` తో కలిపి contract పాటించాలి), `toString()` (readable string representation కోసం).

Professional English Explanation

Every class implicitly extends `Object`, inheriting `equals()`, `hashCode()`, `toString()`, `getClass()`, and others. By default, `equals()` performs reference comparison (same as `==`) and `hashCode()` returns a JVM-derived identity-based value — both are commonly overridden for value-based classes.

Java Code

```
import java.util.Objects;

class Point {
    final int x, y;

    Point(int x, int y) { this.x = x; this.y = y; }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;           // same reference - fast path
        if (!(obj instanceof Point)) return false; // type check
        Point other = (Point) obj;
        return this.x == other.x && this.y == other.y; // content comparison
    }

    @Override
    public int hashCode() {
        return Objects.hash(x, y);               // must be consistent with equals()
    }

    @Override
    public String toString() {
        return "Point(" + x + ", " + y + ")";
    }
}

public class ObjectBasicsDemo {
    public static void main(String[] args) {
        Point p1 = new Point(1, 2);
        Point p2 = new Point(1, 2);

        System.out.println(p1 == p2);           // false - different objects
        System.out.println(p1.equals(p2));       // true - overridden content comparison
        System.out.println(p1.hashCode() == p2.hashCode()); // true - consistent with equals

        System.out.println(p1);                 // uses overridden toString()
        System.out.println(p1.toString());
    }
}
```

Output

```
false
true
true
Point(1, 2)
Point(1, 2)
```

Internal Working / The equals-hashCode Contract

1. If `a.equals(b)` is `true`, then `a.hashCode() == b.hashCode()` **must** be `true`.
2. The reverse is **not** required — equal hash codes don't guarantee `equals()` is `true` (this is a **hash collision**, which is legal and expected).

3. Breaking rule 1 corrupts hash-based collections (`HashMap` , `HashSet`) — an object can "disappear" from a `HashSet` if `hashCode()` isn't consistent with `equals()` . Full internal mechanics (buckets, collisions, treeification) are covered in **Book 05**.

Immutability Intro (deep dive in Book 02/07)

Telugu: `Point` class లో `x` , `y` రెండూ `final` — object create అయిన తర్వాత మారవు. ఇలాంటి **immutable objects** thread-safe గా ఉంటాయి (Book 08) మరియు `HashMap` keys గా సురక్షితంగా వాడొచ్చు (hashcode మారదు కాబట్టి). English: Making `x / y` `final` (and providing no setters) makes `Point` immutable — a foundational pattern that pays off directly in thread-safety (Book 08) and safe use as hash-based collection keys (Book 05).

Real-World Example

Telugu: `Employee` objects ని `HashSet` లో duplicate-check చేయాలంటే, `equals()` / `hashCode()` override చేయకపోతే, ఒకే employee ID ఉన్నా రెండు వేర్వేరు objects గా treat అవుతాయి — data integrity bug. English: Domain objects (`Employee` , `Product` , `Order`) almost always need `equals()` / `hashCode()` overridden by business key (e.g., ID) to behave correctly in collections — a very common real bug when developers forget this.

Interview Answer

"`Object` provides default `equals()` (reference comparison) and `hashCode()` (identity-based). For value-based classes, both should be overridden together, keeping the contract that equal objects must produce equal hash codes — otherwise hash-based collections like `HashMap` / `HashSet` break silently."

Deep Interview Answer

"`hashCode()` consistency with `equals()` is what lets a `HashMap` locate the correct bucket for lookups: it computes the bucket index from `hashCode()` , then uses `equals()` to disambiguate within that bucket. If two equal objects produce different hash codes, a `HashMap` / `HashSet` may store them in different buckets, and lookups by an equal-but-different-object key will silently fail to find an existing entry — a subtle, hard-to-diagnose bug. This is elaborated fully with bucket diagrams in Book 05."

Cross Questions

- Q: What does `Object` 's default `equals()` do? → A: Compares references, identical to `==` .
- Q: Why override `hashCode()` whenever you override `equals()` ? → A: To satisfy the equals-hashCode contract required by hash-based collections.
- Q: What's the risk of only overriding `equals()` but not `hashCode()` ? → A: The class violates the contract; objects that are `.equals()` -equal can land in different hash buckets, breaking `HashMap` / `HashSet` lookups and `contains()` checks.

Tricky Questions

- Q: Can two unequal objects have the same `hashCode()` ? → A: Yes — this is a legal hash collision; the contract only requires the reverse direction (equal → same hash).
- Q: Is it safe to use a **mutable** field in both `equals()` / `hashCode()` for an object stored as a `HashMap` key, then mutate that field afterward? → A: No — this is a real bug: the object's bucket position was computed from the old hash code; after mutation, `map.get(key)` may fail to find it since the recalculated hash won't match its original bucket.

Coding Exercise

L1: Override `equals()`, `hashCode()`, `toString()` for a `Book` class (fields: `isbn`, `title`). **L2:** Demonstrate the "mutable hashCode field" bug: put a mutable object into a `HashSet`, mutate the field used in `hashCode()`, then try `contains()` and observe failure. **L3:** Use `Objects.equals()` and `Objects.hash()` utility methods to simplify the overrides. **L4 (Interview):** State the equals-hashCode contract precisely, in both directions, and explain which direction is mandatory. **L5 (Senior):** Review a domain class using only auto-generated IDE `equals()` / `hashCode()` on a mutable entity used as a `HashMap` key in a long-lived cache — identify the risk and propose a fix (e.g., using an immutable ID-only key). **L6 (Mastery):** Explain, from memory, why hash-based collections require the equals-hashCode contract, connecting it to bucket lookup mechanics (even before reading Book 05 in full).

CHAPTER 16 — Mini Project: Student Management Console App

Goal

Apply every concept from Chapters 1–15 in one cohesive console application — no external libraries, no Collections Framework yet (that's Book 05); only arrays, classes, objects, and fundamentals.

Requirements

1. `Student` class: `id` (int, final), `name` (String), `marks` (int[] for N subjects) — private fields, public getters.
2. Override `equals()`, `hashCode()`, `toString()` on `Student` (by `id`).
3. `StudentManager` class holding a fixed-size `Student[]` array, with methods:
 - `addStudent(Student s)`
 - `findById(int id)` — returns `Student` or `null`
 - `printAll()`
 - `averageMarks(Student s)` — computed from the `marks` array
 - `topScorer()` — loops through all students, returns the one with the highest average
4. An `enum Grade { A, B, C, F }` with a static utility method `fromAverage(double avg)` mapping average marks to a grade.
5. `main()` should: create a `StudentManager`, add 4–5 students (using constructor chaining for a "no marks yet" default), print all students with their grade, and print the top scorer.

Skeleton to Guide You (not the full solution — build it yourself)

```
import java.util.Objects;

class Student {
    private final int id;
    private String name;
    private int[] marks;

    Student(int id, String name, int[] marks) {
        this.id = id;
        this.name = name;
        this.marks = marks;
    }

    Student(int id, String name) {
        this(id, name, new int[0]); // constructor chaining: default = no marks
    }

    int getId() { return id; }
    String getName() { return name; }
    int[] getMarks() { return marks; }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Student)) return false;
        return this.id == ((Student) o).id;
    }

    @Override
    public int hashCode() { return Objects.hash(id); }

    @Override
    public String toString() { return "Student#" + id + " (" + name + ")"; }
}

enum Grade {
    A, B, C, F;

    static Grade fromAverage(double avg) {
        if (avg >= 85) return A;
        if (avg >= 70) return B;
        if (avg >= 50) return C;
        return F;
    }
}

class StudentManager {
    private final Student[] students;
    private int count = 0;

    StudentManager(int capacity) { students = new Student[capacity]; }

    void addStudent(Student s) {
        if (count < students.length) students[count++] = s;
    }

    double averageMarks(Student s) {
        int[] marks = s.getMarks();
```

```

        if (marks.length == 0) return 0.0;
        int total = 0;
        for (int m : marks) total += m;
        return (double) total / marks.length;
    }

    void printAll() {
        for (int i = 0; i < count; i++) {
            Student s = students[i];
            double avg = averageMarks(s);
            System.out.println(s + " avg=" + avg + " grade=" + Grade.fromAverage(avg));
        }
    }

    Student topScorer() {
        Student best = null;
        double bestAvg = -1;
        for (int i = 0; i < count; i++) {
            double avg = averageMarks(students[i]);
            if (avg > bestAvg) { bestAvg = avg; best = students[i]; }
        }
        return best;
    }
}

public class StudentManagementApp {
    public static void main(String[] args) {
        StudentManager manager = new StudentManager(5);
        manager.addStudent(new Student(1, "Anitha", new int[]{90, 85, 92}));
        manager.addStudent(new Student(2, "Kiran", new int[]{60, 55, 70}));
        manager.addStudent(new Student(3, "Ravi", new int[]{40, 35, 50}));

        manager.printAll();
        System.out.println("Top scorer: " + manager.topScorer());
    }
}

```

Expected Output (approx.)

```

Student#1 (Anitha) avg=89.0 grade=A
Student#2 (Kiran) avg=61.66666666666666 grade=C
Student#3 (Ravi) avg=41.666666666666664 grade=F
Top scorer: Student#1 (Anitha)

```

Concepts Reinforced

Classes/objects · constructors + chaining · `this` · access modifiers/encapsulation · arrays · methods (incl. loops) · enums · `equals()` / `hashCode()` / `toString()` · control flow.

Stretch Goals (Level 5/6)

- Add input validation (reject negative marks) — throw a custom check (full exception handling arrives in Book 04).
- Add a `static` field `totalStudentsCreated` tracked across all `Student` objects.

- Make `Student` fully immutable (no setters, defensive copy of the `marks` array in the constructor and getter) — connects directly to the immutability discussion in Chapter 15.
-

FINAL REVISION NOTES

- **JVM/JDK/JRE:** JDK $\supset$ JRE $\supset$ JVM. Bytecode is the WORA contract.
 - **Primitives vs References:** value vs pointer semantics; stack vs heap.
 - **`==` vs `.equals()`:** reference identity vs content equality (override `equals()` / `hashCode()` together).
 - **Integer division & short-circuit:** `int/int` truncates; `&&` / `||` skip unnecessary evaluation.
 - **switch fall-through:** classic legacy bug; arrow-form `switch` (Java 14+) avoids it.
 - **Arrays:** fixed-size, heap-allocated objects, $O(1)$ index access, bounds-checked at runtime.
 - **Pass-by-value always:** object "pass-by-reference" behavior is really "the reference value is copied."
 - **Constructors:** no return type, chaining via `this(...)` must be first statement; default constructor disappears once you define any constructor.
 - **static:** per-class, not per-object; can't access instance members directly.
 - **final:** locks variable reassignment / prevents override / prevents subclassing — doesn't make referenced objects immutable by itself.
 - **Access modifiers:** `private` < default < `protected` < `public`, in increasing visibility.
 - **String immutability:** enables pooling, safe hashing, thread-safety; use `StringBuilder` in loops.
 - **Integer cache:** -128..127 autoboxed values are `==`-equal; beyond that, always use `.equals()`.
 - **Enums:** type-safe constants, full classes internally, can't extend classes but can implement interfaces.
 - **Nested classes:** static nested (no outer ref) vs inner (implicit outer ref) vs local vs anonymous.
 - **equals-hashCode contract:** equal objects MUST share hash code; the reverse isn't required.
-



CHEAT SHEET

Primitives: byte short int long float double char boolean
Default field values: numeric=0, boolean=false, reference=null (locals: NO default)
== : reference/value identity | .equals(): content equality (override together with hashCode)
String pool: literals reused; new String() forces new heap object
StringBuilder: mutable, fast, NOT thread-safe | StringBuffer: mutable, thread-safe (synchronized)
Access modifiers (most -> least restrictive): private -> default -> protected -> public
static: per-class | instance: per-object
final variable: no reassignment | final method: no override | final class: no subclass
Constructor: same name as class, no return type, this(...)/super(...) must be first line
Arrays: fixed size, heap object, bounds-checked, default null/0 per element type
Enum: type-safe fixed constants, extends java.lang.Enum implicitly, can implement interfaces
Autoboxing: int -> Integer (compiler sugar) | Unboxing null -> NullPointerException risk
switch (traditional): needs break to avoid fall-through | switch expression (->): no fall-through, returns a value



INTERVIEW QUESTION BANK — Java Fundamentals

Beginner

1. What is JVM/JDK/JRE and how do they relate?
2. Difference between `==` and `.equals()` ?
3. What are Java's 8 primitive types?
4. Why is Java called "platform-independent"?
5. What is the default value of an uninitialized local variable?

Intermediate 6. Why is `String` immutable, and what problems would mutability cause? 7. Explain the Integer Cache and why `Integer a=127,b=127; a==b` differs from `200==200` . 8. What is constructor chaining, and why must `this()` / `super()` be the first statement? 9. Difference between `StringBuilder` and `StringBuffer` ? 10. Explain the equals-hashCode contract and why it matters for `HashMap` / `HashSet` .

Advanced 11. Why does unboxing a `null` wrapper throw `NullPointerException` , and where have you seen this in real code? 12. Explain why arrays are covariant and how that leads to `ArrayStoreException` . 13. Why can't `static` methods be truly overridden (only hidden)? 14. What's the actual memory difference between a `final` reference and a truly immutable object? 15. Why do frameworks like Hibernate require a no-arg constructor on entity classes?

Senior/Architect 16. Design a `Money` type avoiding `double` rounding issues for a payments system — what trade-offs do you consider? 17. A `HashSet` "loses" an object after some field used in `hashCode()` was mutated — diagnose and fix the design. 18. When would you deliberately choose `StringBuffer` over `StringBuilder` in a modern concurrent design, if ever? 19. Explain how you'd model a finite state machine (order lifecycle) using enums with per-constant behavior. 20. Walk through, end-to-end, what happens physically (stack/heap/method area) when `new Employee("Suresh", 5000)` executes.

(Short/Professional/Deep-Senior answer format for every question, organized by full seniority ladder, is provided at scale in Book 23 — Java Interview Master Book, which aggregates and cross-links questions from every book in this series.)



CROSS-QUESTION ENGINE — Sample Chains

Q: What is `String`? → Q: Why is it immutable? → Q: What problem does immutability solve for `HashMap` keys? → Q: What happens if you use a mutable object as a `HashMap` key instead? → Q: How would you fix that design?

Q: What is `static`? → Q: Where does static data live in memory? → Q: Why can't static methods access instance fields? → Q: What's the risk of static mutable state in a multi-threaded app? (bridges to Book 08)

Q: What is a constructor? → Q: What happens if you don't define one? → Q: Can you overload constructors? → Q: What is constructor chaining and its restriction? → Q: Why do ORMs require a no-arg constructor?

(All chains above are answered inline in the relevant chapters — re-read Chapters 9, 11, 15 if any link feels shaky.)



CONSOLIDATED EXERCISES (All Levels)

Level 1 — Beginner: Re-do each chapter's L1 exercise without looking at the book. **Level 2 — Intermediate:** Re-do each chapter's L2 exercise, then explain out loud (or in Telugu) why each line works. **Level 3 — Advanced:** Combine Chapters 5–9 into one program: an array of objects, each with a constructor, `static` counter, and `final` ID field. **Level 4 — Interview:** Answer all 20 questions in the Interview Question Bank from memory, timing yourself (aim: <90 seconds each for Beginner/Intermediate). **Level 5 — Senior:** Complete the Chapter 16 Mini Project stretch goals (validation, static counter, full immutability). **Level 6 — Mastery:** Teach Chapters 9, 11, and 15 out loud to someone else (or record yourself) using only the diagrams you draw from memory — no notes.



ONE-DAY REVISION PLAN (≈4–5 hours)

Time	Focus
0:00–0:30	Ch.1–2: JVM/JDK/JRE, variables/data types — read Cheat Sheet first, then skim chapters
0:30–1:00	Ch.3–4: Operators, control flow — do L1/L2 exercises
1:00–1:30	Ch.5–6: Arrays, methods — do L2/L3 exercises
1:30–2:15	Ch.7–9: Classes/objects, constructors, static/final — this is the highest-density block, don't rush
2:15–2:30	Break
2:30–3:00	Ch.10–11: Access modifiers, String family
3:00–3:30	Ch.12–14: Wrapper classes, enums, nested classes
3:30–4:00	Ch.15: equals/hashCode/toString — re-read the contract twice
4:00–4:45	Build the Chapter 16 mini project from scratch, unaided
4:45–5:00	Answer the full Interview Question Bank from memory



ONE-WEEK MASTER REVISION PLAN

Day	Focus
1	Ch.1–4 (setup, variables, operators, control flow) — code every example yourself
2	Ch.5–6 (arrays, methods) — do all 6 exercise levels for each
3	Ch.7–9 (classes, objects, constructors, static/final) — the OOP-foundation core; re-draw all memory diagrams from scratch
4	Ch.10–12 (access modifiers, String family, wrappers) — focus on the interview traps (immutability, caching, null-unboxing)
5	Ch.13–15 (enums, nested classes, Object basics) — implement the equals-hashCode contract in 3 different classes
6	Build the Chapter 16 mini project + all stretch goals; write your own additional mini-project (e.g., a <code>Library</code> system) reusing every concept
7	Full mock interview: answer all 20 bank questions + all cross-question chains out loud, in both Telugu and English, without notes

Note: this plan builds fast, durable *recall*. Real 10-years-equivalent judgment comes from writing production code over time — this book gives you the correct foundation to build that experience efficiently, not a shortcut around it.



FINAL MASTERY CHECKLIST

- ☐ I can explain JVM/JDK/JRE and the compile-execute pipeline without notes.
- ☐ I can explain `==` vs `.equals()` and why both matter for every reference type.
- ☐ I can explain stack vs heap for primitives, references, and objects, with a diagram.
- ☐ I can explain integer division, short-circuit evaluation, and switch fall-through with examples.
- ☐ I can explain arrays' fixed-size nature, bounds checking, and covariance risk.
- ☐ I can explain pass-by-value semantics for both primitives and object references.
- ☐ I can write and explain constructor chaining and the default-constructor rule.
- ☐ I can explain `static` vs instance state and where each lives in memory.
- ☐ I can explain `final` on variables, methods, and classes, including the "final reference, mutable content" trap.
- ☐ I can explain all four access modifiers with concrete cross-package/cross-class scenarios.
- ☐ I can explain String immutability, the String Pool, and when to use `StringBuilder` / `StringBuffer`.
- ☐ I can explain autoboxing/unboxing, the Integer Cache, and the null-unboxing NPE trap.
- ☐ I can design an enum with fields, constructors, and per-constant behavior.
- ☐ I can distinguish static nested, inner, local, and anonymous classes and when to use each.
- ☐ I can state and apply the equals-hashCode contract correctly.
- ☐ I built the Chapter 16 mini project unaided, including at least one stretch goal.
- ☐ I answered the full Interview Question Bank confidently in both Telugu and English.

Once every box is checked, you are ready for `02_Java_00PS_Mastery_Telugu_English.md`.